

《五子棋编程对战》项目作业报告

基于 Qt Widgets 的编程题库驱动式五子棋学习游戏

项目名称	五子棋编程对战
开发环境	Qt / C++ / JSON / TCP Socket
核心功能	本地对战、编程答题、技能机制、局域网联网、错题复盘与 JSON 导出
报告内容	程序功能介绍、模块与类设计、小组成员分工、项目总结与反思

摘要

本项目以传统五子棋为基础，将编程知识问答嵌入对局过程，形成一个兼具学习性和对抗性的桌面应用。玩家在落子前需要完成题目，答题结果会影响落子机会和技能资源；对局结束后，系统汇总双方错题，既能导出 JSON 文件，也能在程序内按答题页面逐题复盘。

从实现角度看，项目采用 Qt Widgets 构建界面，使用 C++ 完成规则、流程和网络通信，使用 JSON 保存题库和复盘数据。整体功能覆盖菜单选择、棋盘交互、题目筛选、技能触发、联网同步、胜负结算和错题回看，形成了比较完整的课程项目闭环。

一、程序功能介绍

1. 项目定位与使用场景

《五子棋编程对战》不是单纯的棋类程序，而是“游戏化编程训练”的实践项目。它将五子棋清晰直观的对战规则与程序设计题目结合起来，使玩家在对抗过程中不断完成知识点练习。

项目适合用于课程展示、小组练习和课后复习。课堂中可以通过联网对战展示程序综合能力；课后可以通过本地对战反复练习；对局结束后的错题复盘则能帮助玩家找到薄弱知识点。

2. 本地对战功能

本地对战面向同一台电脑上的两名玩家。程序初始化十五路棋盘，黑白双方轮流行动，界面左右两侧显示玩家身份、技能按钮、时间和能量，中间区域显示棋盘。玩家完成答题后才能在合法位置落子，程序会自动判断是否形成五子连珠。

主菜单提供难度选择和知识点选择，使对局可以围绕不同训练范围展开。开始按钮被区分为“本地对战”和“联网对战”，避免玩家混淆不同模式。

3. 答题落子功能

答题落子是项目的核心交互。玩家行动时弹出题目窗口，题目包含题干、选项和提交操作。答对后允许落子，答错则记录错题并按照规则处理。这样，棋盘上的每一步都与知识掌握情况相关联，避免游戏和学习内容割裂。

题目筛选由难度和知识点共同决定。程序从 questions.json 中读取题目后，根据玩家选择生成题目池，从而保证训练内容更有针对性。

4. 技能与能量机制

为提升策略性，项目加入了能量和技能系统。玩家可消耗能量使用跳过答题、消除红叉、排除错误选项、换一道题、延长答题时间等技能。技能既能辅助答题，也会影响对局节奏，因此玩家需要在保留资源和争取落子机会之间做选择。

技能机制的实现需要考虑多个状态：当前玩家是谁、能量是否足够、技能是否可以在当前阶段触发、触发后是否需要影响答题窗口或棋盘显示。该设计让程序不再只是静态棋盘，而具有更强的互动性。

5. 局域网联网对战

联网对战采用 TCP 局域网房间模式。一方创建房间并监听端口，另一方输入房主 IP 和端口加入。连接建立后，双方通过约定消息同步准备状态、题目、落子、技能和结束信息，保证两台电脑上的棋盘状态一致。

该方案适合同一校园网、实验室网络或 VPN 能够互通的场景。相比互联网大厅和账号系统，局域网 TCP 房间版实现难度适中，符合课程项目的时间和规模要求。

6. 对局结束与错题复盘

对局结束后，程序会汇总本局双方出现过的错题。系统保留 JSON 导出，便于后续检查和题库维护；同时提供可视化复盘页面，使玩家像答题时一样逐题查看错题、选项和正确答案，并支持上一题、下一题切换。

复盘功能使项目从“完成一局游戏”延伸到“完成一次学习反馈”。这也是本项目区别于普通五子棋程序的重要价值。

二、项目各模块与类设计细节

1. 总体架构

项目采用相对清晰的分层结构：界面层负责显示和交互，规则层负责棋盘状态与胜负判断，题库层负责题目数据读取和筛选，网络层负责局域网通信，复盘层负责错题收集和展示。各层通过函数调用和 Qt 信号槽机制协作。

模块/类	主要职责	设计细节
MainWindow	主窗口、菜单、棋盘、技能、联网和复盘流程	负责把用户操作、题库、规则模型和网络事件串成完整对局流程。
GameModel	棋盘数据、落子合法性、胜负判断	保存十五路棋盘状态，并在横、竖、斜方向检查连续棋子数量。
QuestionDialog	题目展示、选项选择、答题提交	封装答题交互，使题目窗口与棋盘绘制逻辑分离。
NetworkManager	创建房间、加入房间、TCP 消息收发	用信号槽把网络事件传递给主窗口处理，避免网络代码直接控制 UI。
questions.json	题库数据和题目属性	以结构化方式保存题干、选项、答案、难度和知识点，便于维护。

2. GameModel：规则模型

GameModel 是五子棋规则的核心。它不依赖具体按钮和窗口，而是专注于棋盘数据本身，包括当前位置是否为空、落子后应该轮到哪一方、是否出现五子连珠等。规则模型独立后，本地对战和联网对战都可以复用同一套判断逻辑。

3. QuestionDialog：答题窗口

QuestionDialog 将题库中的一条题目转化为玩家可操作的界面。它需要保证题干和选项显示清楚，提交后能返回选择结果，并配合倒计时、换题、排除错误选项等技能效果。答题窗口的稳定性直接影响玩家对项目质量的感受。

4. MainWindow：流程控制中心

MainWindow 是项目中职责最综合的部分。它负责主菜单、难度与知识点选择、本地对战入口、联网对战入口、棋盘绘制、状态区、技能按钮、复盘页面等界面内容，也负责控制从开始对局到结束复盘的完整流程。

该类的设计难点在于状态一致性。例如当前是本地还是联网模式、是否轮到本机、题目是否已经提交、技能是否生效、棋盘是否允许点击、对局是否结束等，都需要在不同事件发生后保持正确。

5. NetworkManager：TCP 联网通信

NetworkManager 将 TCP 连接细节从主界面中抽离。房主端创建服务器并监听端口，加入端主动连接房主地址。连接成功后，双方通过统一格式的消息传递对局事件，例如准备、落子、技能使用、重新开始和退出房间。

网络模块只负责“把消息送到”和“把收到的消息通知出去”，真正如何改变棋盘和界面仍由 MainWindow 处理。这种设计可以降低耦合度，也便于以后替换为互联网服务器方案。

6. 数据与复盘设计

questions.json 保存题库，复盘 JSON 保存本局错题。二者都使用结构化数据格式，优点是清晰、易扩展、便于人工检查。题库维护时可以增加题目查重、选项校验和答案规范化；复盘数据则可以继续扩展为错题统计或学习报告。

关键流程	流程说明
答题落子	玩家触发行动后弹出题目，提交答案；答对则调用规则模型落子，答错则记录错题并给出反馈。
技能使用	检查当前玩家能量和技能条件，执行效果后更新界面，联网模式下还需要同步给对方。
联网同步	本机操作被转换为 TCP 消息发送，对方收到后还原为对应操作，双方共同维护一致棋盘。
错题复盘	对局结束时汇总双方错题，写入 JSON，同时打开可视化复盘页面逐题查看。

三、小组成员分工情况

本项目按照规则实现、界面维护、题库与联网三个方向进行分工。这样的分工能够让每位成员有明确负责范围，同时又需要在后期进行整体联调，保证规则、界面、题库和网络功能能够在同一套对局流程中协同工作。

成员	主要分工	具体说明
唐梓涵	游戏规则及技能实现	负责五子棋基础规则、胜负判断、落子逻辑和技能效果设计，包括技能消耗、触发条件以及技能对答题或棋局的影响。
尹致君	游戏 UI 界面维护	负责主菜单、棋盘界面、状态显示、按钮布局、答题窗口和复盘页面等界面维护，使应用布局稳定、信息清晰。
李佳旺	题库及联网对战机制	负责 questions.json 题库整理、题目筛选、错题记录，以及 TCP 局域网房间联机机制，包括创建房间、加入房间、消息同步和联机调试。

协作方式

三位成员的工作在开发过程中存在明显交叉。例如技能按钮需要 UI 支持，技能效果又需要规则逻辑配合；题库答题需要 QuestionDialog 显示，也需要 MainWindow 控制落子流程；联网对战不仅要发送棋盘位置，还要同步题目、技能和结算状态。

因此，项目后期的重点是联调和测试。每个模块单独能运行并不代表完整对局稳定，只有把本地对战、联网对战、答题、技能和复盘全部串起来测试，才能发现状态错位、按钮无效、网络不同步和题库异常等问题。

测试与调试

调试过程中重点关注了界面错位、题库重复选项、技能点击无效、联网端口连接失败、防火墙拦截、错题导出和复盘翻页等问题。这些问题覆盖了界面、数据、规则和环境多个方面，也体现了桌面应用开发中“代码正确”和“实际可用”之间仍需要大量验证。

四、项目总结与反思

1. 项目成果

本项目完成了一个较完整的 Qt 桌面游戏原型。它不仅实现了五子棋基础玩法，还加入了编程题库、答题落子、技能系统、本地对战、局域网联网对战和错题复盘功能。相比只实现棋盘绘制，项目具有更完整的用户流程和学习价值。

从技术实践角度看，项目综合使用了 C++ 面向对象设计、Qt 信号槽、JSON 数据处理、计时器、界面布局和 TCP Socket 通信。每个模块都有明确作用，最终通过真实对局连接成完整应用。

2. 问题与不足

开发中暴露出一些不足：界面在不同尺寸下容易出现错位；题库质量需要持续维护，例如重复选项和答案表述不统一；联网对战容易受到防火墙、IP 网段、端口监听和校园网策略影响；本地逻辑与联网同步之间也需要更严格的状态管理。

这些问题说明，课程项目不能只关注功能是否写出，还要关注数据质量、异常提示和真实运行环境。尤其是联网功能，即使代码逻辑没有明显问题，也可能因为网络不可达或防火墙拦截导致连接失败。

3. 后续改进方向

后续可以继续改进三个方向。第一，建设题库校验工具，自动检查重复选项、答案缺失、知识点为空等问题。第二，优化联网体验，增加连接诊断、断线提示、重连机制和更直观的房间状态。第三，继续完善 UI 适配，让主菜单、联网页面、棋盘页面和复盘页面在不同分辨率下保持整齐。

如果未来升级到互联网对战，则需要引入公网服务器或云端中转服务，进一步处理账号、房间列表、NAT 穿透、断线恢复和安全校验等问题。难度会明显高于局域网版本，但也能让项目更接近真实产品。

4. 总体反思

通过本项目，小组成员认识到模块化设计和持续联调的重要性。规则、UI、题库和网络看似独立，但用户体验来自完整流程。只有每个环节都稳定，项目才能真正可玩、可学、可展示。总体来看，本项目达到了课程实践目标，也为后续扩展提供了清晰基础。